

- With enough capacity, models can perform perfectly on training data
 - ↳ Does NOT mean it will generalize well to testing data
- Test errors have 3 distinct causes:
 - 1) Inherent uncertainty in the task
 - 2) The amount of training data
 - 3) The choice of model
 - ↳ Contingent on hyperparameter search
- Dictates how to select model ; optimizer hyperparams.

8.1: Training a Simple Model:

- Were using MNIST-1D to explore performance.

↳ Has 10 classes $\{y \in \{0, \dots, 9\}\}$

↳ Represents digits 1-9

↳ Full training dataset $\{x_i, y_i\}$ has $I = 4000$ training examples, each consists of $D_i = 40$ dimensions representing horizontal offset at 40 positions.

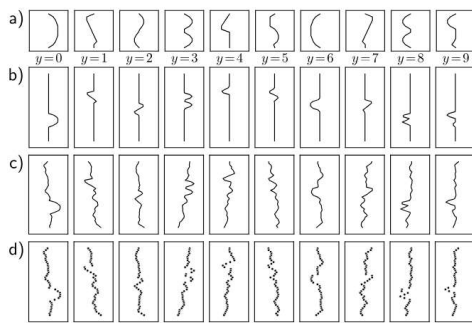


Figure 8.1 MNIST-1D. a) Templates for 10 classes $y \in \{0, \dots, 9\}$, based on digits 0-9. b) Training examples x are created by randomly transforming a template and c) adding noise. d) The horizontal offset of the transformed template is then sampled at 40 vertical positions. Adapted from (Greydanus, 2020)

- We use a network with:

↳ $D_i = 40$ inputs

↳ $D_o = 10$ outputs

↳ Outputs passed through Softmax to get class probabilities

↳ 2 hidden layers, with $D = 100$ hidden units in each

↳ Trained using SGD w/ batch size 100, $LR = 0.1$, for 150 epochs

↳ Multiclass cross-entropy loss is used

- Training gives the results:

↳ Training error goes to 0

↳ Loss approaches 0

- This isn't the full picture!

- This isn't the full picture!

- To estimate true performance, we need a separate test set of input/output pairs $\{x_i, y_i\}$.
 ↳ We generate 1,000 more examples from the same process as training.

- Results:

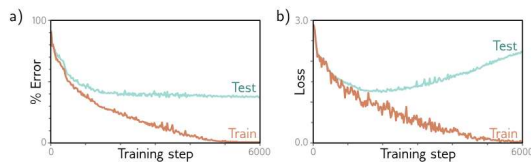


Figure 8.2 MNIST-1D results. a) Percent classification error as a function of the training step. The training set errors decrease to zero, but the test errors do not drop below $\sim 40\%$. This model doesn't generalize well to new test data. b) Loss as a function of the training step. The training loss decreases steadily toward zero. The test loss decreases at first but subsequently increases as the model becomes increasingly confident about its (wrong) predictions.

- Testing error decreases, but only to $\sim 40\%$. [better than random chance error of 90-1.]

- Still significantly worse than the training error

- ↳ The model has NOT generalized well.

- Notice, loss goes up after a while, while error stays constant;

- ↳ This means the model makes the same mistakes but with increasing confidence

- ↳ so probability of the correct answer goes down, increasing negative log-likelihood

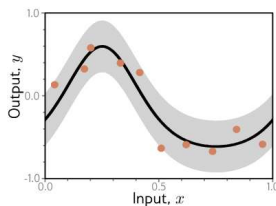
- ↳ [Increasing confidence is a side effect of using Softmax; pre-activations are driven to extreme values to make the probability of training data go to one.]

8.2: Sources of Error:

- Why did the model fail to generalize?

- For visualization, consider 1D- Least squares regression where we know how the ground truth data were generated.

Figure 8.3 Regression function. Solid black line shows ground truth function. To generate I training examples $\{x_i, y_i\}$, the input space $x \in [0, 1]$ is divided into I equal segments and one sample x_i is drawn from a uniform distribution within each segment. The corresponding value y_i is created by evaluating the function at x_i and adding Gaussian noise (gray region shows ± 2 standard deviations). The test data are generated in the same way.



- We fit a simple shallow network to this data:

- ↳ weights; bins are chosen so that the "joints" are evenly spaced.

- ↳ If there are D hidden units, the joints are at $0, 1/D, 2/D, \dots, D-1/D$.

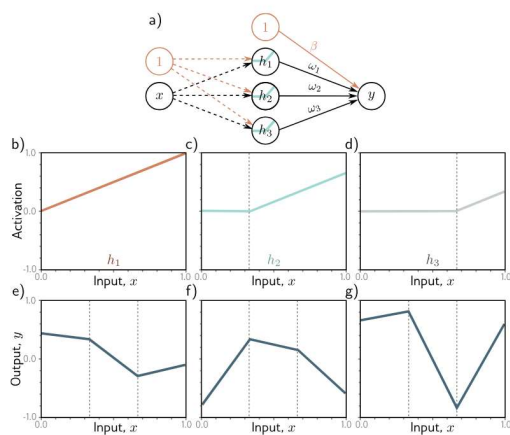


Figure 8.4 Simplified neural network with three hidden units. a) The weights and biases between the input and hidden layer are fixed (dashed arrows). b-d) They are chosen so that the hidden unit activations have slope one, and their joints are equally spaced across the interval, with joints at $x = 0, x = 1/3,$ and $x = 2/3,$ respectively. Modifying the remaining parameters $\phi = \{\beta, \omega_1, \omega_2, \omega_3\}$ can create any piecewise linear function over $x \in [0, 1]$ with joints at $1/3$ and $2/3$. e-g) Three example functions with different values of the parameters ϕ .

Noise, Bias, & Variance:

1) Noise:

- The data generation process includes noise, so there are multiple possible valid outputs y for each input x .
- This source of error is difficult to overcome for the testing data.
- This doesn't limit training performance.
- Noise can arise because of genuine stochastic aspects to the data generation process, OR because the data was mislabeled, OR there are further explanatory variables not observed.
- In rare cases, there is no noise. Ex: approximating a deterministic function that is hard to compute.
- Usually a fundamental limitation on possible test performance.

2) Bias:

- The model is not flexible enough to fit the true function
- Our shallow network can't approximate the sinusoidal function with just 3 regions
- This happens even if the parameters are chosen optimally.

3) Variance:

- Training data is limited, so there isn't a way to tell systematic changes in the true function to noise in the training data.
- So when we fit a model, we don't end up with the closest approximation to the true function.
- For different training sets, the results will be slightly different each time
- Variance could also come from stochastic optimizers, if they don't converge to the same solution each time.

• In different training sets, the results will be slightly different each time

• Variance could also come from stochastic optimizers, if they don't converge to the same solution each time.

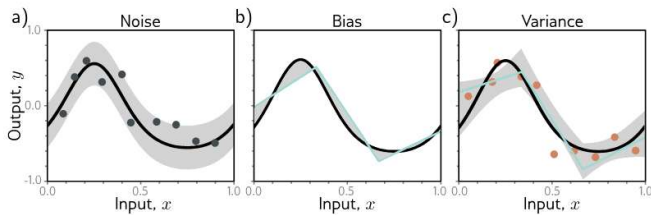


Figure 8.5 Sources of test error. a) Noise. Data generation is noisy, so even if the model exactly replicates the true underlying function (black line), the noise in the test data (gray points) means that some error will remain (gray region represents two standard deviations). b) Bias. Even with the best possible parameters, the three-region model (cyan line) cannot exactly fit the true function (black line). This bias is another source of error (gray regions represent signed error). c) Variance. In practice, we have limited noisy training data (orange points). When we fit the model, we don't recover the best possible function from panel (b) but a slightly different function (cyan line) that reflects idiosyncrasies of the training data. This provides an additional source of error (gray region represents two standard deviations). Figure 8.6 shows how this region was calculated.

Mathematical Formulation of test error:

• We can make precise definitions for noise, bias, & variance:

• Consider a 1D regression problem where the data generation process has additive noise with variance σ^2 .

↳ So there is different outputs y for the same input x .

For each x there is a distribution $\text{Pr}(y|x)$ with the expected value/mean $\mu(x)$:

$$\mu(x) = E_y[y(x)] = \int y(x) \text{Pr}(y|x) dy$$

$$\text{and fixed noise } \sigma^2 = E_y[(\mu(x) - y(x))^2]$$

Now consider a least squares loss between Model Prediction $f(x, \theta)$ at position x

& the observed value $y(x)$ at that position:

$$\begin{aligned} L(x) &= (f(x, \theta) - y(x))^2 \\ &= ((f(x, \theta) - \mu(x)) + (\mu(x) - y(x)))^2 \\ &= (f(x, \theta) - \mu(x))^2 + 2(f(x, \theta) - \mu(x))(\mu(x) - y(x)) + (\mu(x) - y(x))^2 \end{aligned}$$

• The underlying function is stochastic, so loss depends on the particular $y(x)$ observed:

$$\begin{aligned} E[L(x)] &= E_y[(f(x, \theta) - \mu(x))^2 + 2(f(x, \theta) - \mu(x))(\mu(x) - y(x)) + (\mu(x) - y(x))^2] \\ &= (f(x, \theta) - \mu(x))^2 + \sigma^2 \end{aligned}$$

↳ After a lot of algebra & rules of expectations

- The loss can be broken down into 2 terms; squared deviation between the model & true function mean and the second term is noise.
- First term can be split even further:
 ↳ Parameters θ of the model fit depend on the training dataset D .

With even more algebra, we arrive at:

$$E_D [E_y [LL(x)]] = E_D \left[\underbrace{(f(x, \theta[D]) - f_{\mu}(x))^2}_{\text{Variance}} + \underbrace{(f_{\mu}(x) - \mu(x))^2}_{\text{Bias}} + \underbrace{\sigma^2}_{\text{Noise}} \right]$$

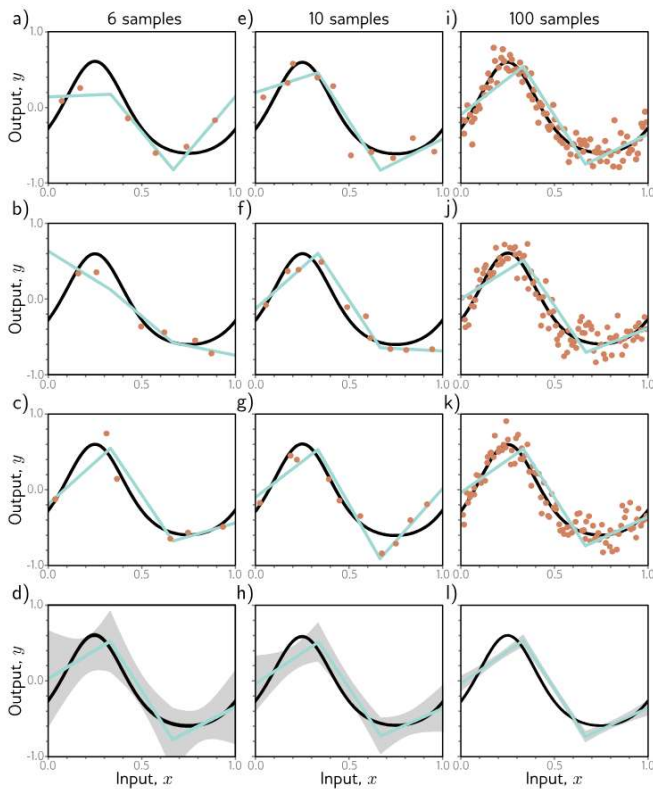
- Variance, bias, & noise combine additively for expected loss, for regression tasks.

8.3: Reducing Error:

- Test error comes from noise, bias, & variance.
- Noise is insurmountable; **nothing we can do to circumvent it**
 ↳ Represents a fundamental limit on model performance
- We can reduce bias & variance!

Reducing Variance:

- Variance results from limited, noisy training data
 ↳ Fitting on 2 different training sets results in slightly different parameters
- We can reduce variance by increasing the quantity of training data.
 ↳ Averages out inherent noise
 ↳ Ensures the input space is well sampled



With only 6 data points, the variance is significant. We end up with very different fits

As we increase train data, variance reduces; the fit models become similar.

Figure 8.6 Reducing variance by increasing training data. a-c) The three-region model fitted to three different randomly sampled datasets of six points. The fitted model is quite different each time. d) We repeat this experiment many times and plot the mean model predictions (cyan line) and the variance of the model predictions (gray area shows two standard deviations). e-h) We do the same experiment, but this time with datasets of size ten. The variance of the predictions is reduced. i-l) We repeat this experiment with datasets of size 100. Now the fitted model is always similar, and the variance is small.

Reducing Bias:

- Bias comes from the model's inability to describe the true underlying function
- We can increase model capacity to reduce bias
 - ↳ i.e. adding more hidden units or layers.

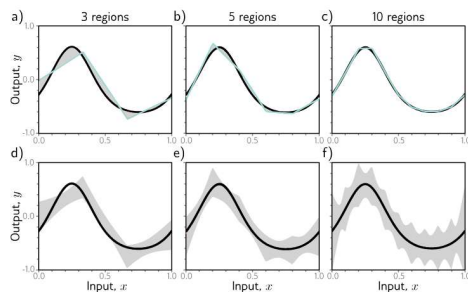


Figure 8.7 Bias and variance as a function of model capacity. a-c) As we increase the number of hidden units of the toy model, the number of linear regions increases, and the model becomes able to fit the true function closely; the bias (gray region) decreases. d-f) Unfortunately, increasing the model capacity has the side-effect of increasing the variance term (gray region). This is known as the bias-variance trade-off.

Bias - Variance Tradeoff:

- For a fixed training dataset, the variance typically **increases** as model capacity increases.
 - ↳ so increasing capacity does not necessarily reduce test error.
 - ↳ known as the bias-variance tradeoff

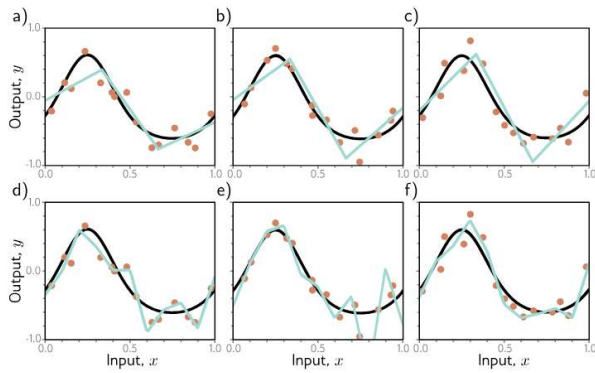
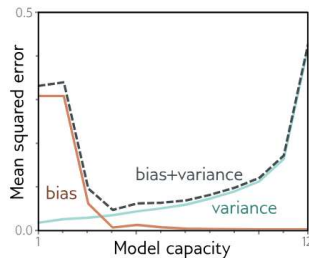


Figure 8.8 Overfitting. a-c) A model with three regions is fit to three different datasets of fifteen points each. The result is similar in all three cases (i.e., the variance is low). d-f) A model with ten regions is fit to the same datasets. The additional flexibility does not necessarily produce better predictions. While these three models each describe the training data better, they are not necessarily closer to the true underlying function (black curve). Instead, they overfit the data and describe the noise, and the variance (difference between fitted curves) is larger.

The extra model capacity fits to the noise of the dataset. We are overfitting!

- So when we add capacity to the model, bias decreases but the variance increases for a fixed training set.
- ↳ Suggests there's an optimal capacity where the bias is not too large & the variance is relatively large.

Figure 8.9 Bias-variance trade-off. The bias and variance terms from equation 8.7 are plotted as a function of the model capacity (number of hidden units / linear regions in range of data) in the simplified model using training data from figure 8.8. As the capacity increases, the bias (solid orange line) decreases, but the variance (solid cyan line) increases. The sum of these two terms (dashed gray line) is minimized when the capacity is four.



8.4: Double Descent:

• Going back to MNIST-1D

- We use 10,000 training examples, test with another 5,000 examples.
- Analyzing the training / testing performance as we increase model capacity.
- Training is w/ Adam ; a step size of 0.005 using a full batch of 10,000 examples for 4,000 steps.
- We can see that the training error decreases as the model capacity grows ; quickly goes to 0.
- ↳ Vertical dashed line is the capacity where the model has the same # of training parameters as training examples.
- The test error decreases as we add capacity, ; keep decreasing [Under the bias-variance tradeoff, it should increase again]
- We repeat the experiment, but this time randomize 15% of the training labels.
- ↳ Training error goes to 0, but there is more randomness. The model learns almost as many parameters as there are data points to memorize the data.
- ↳ The test error DOES show the bias-variance trade off
- ↳ However: The test loss starts to decrease again! And goes lower than it did when it was initially decreasing.
- ↳ Known as double descent!

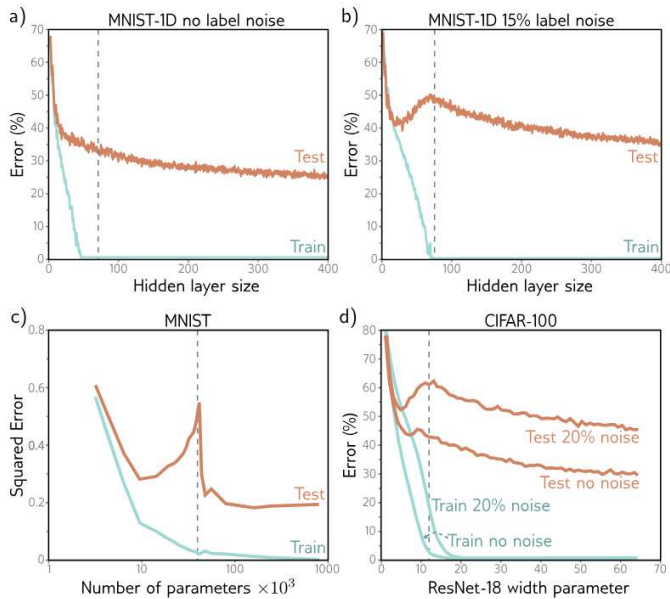


Figure 8.10 Double descent. a) Training and test error on MNIST-1D for a two-hidden layer network as we increase the number of hidden units (and hence parameters) in each layer. The training error decreases to zero as the number of parameters approaches the number of training examples (vertical dashed line). The test error does not show the expected bias-variance trade-off but continues to decrease even after the model has memorized the dataset. b) The same experiment is repeated with noisier training data. Again, the training error reduces to zero, although it now takes almost as many parameters as training points to memorize the dataset. The test error shows the predicted bias/variance trade-off; it decreases as the capacity increases but then increases again as we near the point where the training data is exactly memorized. However, it subsequently decreases again and ultimately reaches a better performance level. This is known as double descent. Depending on the loss function, the model, and the amount of noise in the data, the double descent pattern can be seen to a greater or lesser degree across many datasets. c) Results on MNIST (without label noise) with shallow neural network from Belkin et al. (2019). d) Results on CIFAR-100 with ResNet18 network (see chapter 11) from Nakkiran et al. (2021). See original papers for details.

- The Phenomenon can be seen across datasets, it emerges sometimes when adding noise to the labels
- The first part of the curve is called the **Classical / Under-Parametrized regime**.
- The second part is called the **Modern / Over-Parametrized regime**.
- The central part where the error term increases is the **critical regime**.

Explanation:

- The discovery of double descent is recent, it's puzzling.
- It results from the interaction of 2 Phenomena:
 - 1) The test performance gets temporarily worse when the model has just enough capacity to memorize the data.
 - 2) The test performance continues to improve w/ capacity even when this exceeds the point where the training data are all classified correctly.
- #1 is explained by the bias-variance trade off
- #2 is confusing. It's unclear why performance is better in the over-Parametrized regime.
 - ↳ There aren't enough data points to constrain the model parameters uniquely.
- To understand why performance increases as we add more parameters, note that once the model has enough

- To understand why performance increases as we add more parameters, note that once the model has enough capacity to drive the training loss to ~ 0 , the model fits the training data almost perfectly.
 - ↳ Implies that further capacity can't help the model fit the training data any better.
 - ↳ Any change must occur between the training points!
 - ↳ The tendency of a model to prioritize one solution over another between data points is known as its inductive bias!
- The model's behavior between data points is important because in high-dimensional space, the training data are extremely sparse.
 - ↳ The tendency of the volume of high dimensional space to overwhelm the number of training points is called **the curse of dimensionality**.
- The implication is that problems in high-dimensions have only small regions of the input space where we observe data; large gaps in between.
- The assumed explanation for double descent is that as we add capacity to the model, it interpolates between the nearest data points increasingly smoothly.
 - ↳ In the absence of information about what happens between the training points, assuming smoothness is sensible; it will probably generalize reasonably to new data.

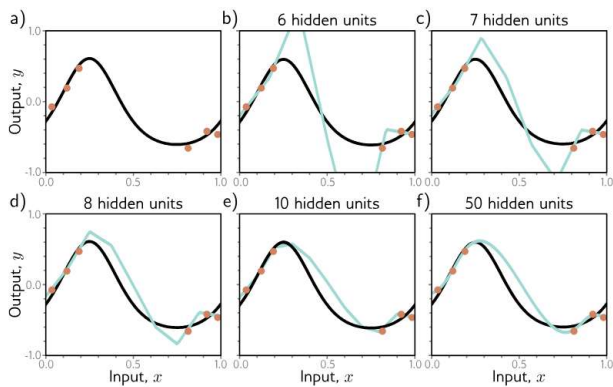


Figure 8.11 Increasing capacity (hidden units) allows smoother interpolation between sparse data points. a) Consider this situation where the training data (orange circles) are sparse; there is a large region in the center with no data examples to constrain the model to mimic the true function (black curve). b) If we fit a model with just enough capacity to fit the training data (cyan curve), then it has to contort itself to pass through the training data, and the output predictions will not be smooth. c-f) However, as we add more hidden units, the model has the ability to interpolate between the points more smoothly (smoothest possible curve plotted in each case). However, unlike in this figure, it is not obliged to.

- But, this does not explain why over-parameterized models should produce smooth functions.
 - ↳ If the modern regime of double descent is explained by increasing smoothness, then what exactly is encouraging that smoothness?

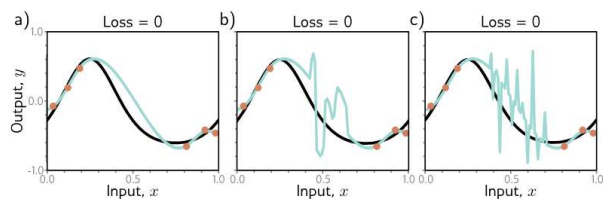


Figure 8.12 Regularization. a-c) Each of the three fitted curves passes through the data points exactly, so the training loss for each is zero. However, we might expect the smooth curve in panel (a) to generalize much better to new data than the erratic curves in panels (b) and (c). Any factor that biases a model toward a subset of the solutions with a similar training loss is known as a regularizer. It is thought that the initialization and/or fitting of neural networks have an implicit regularizing effect. Consequently, in the over-parameterized regime, more reasonable solutions, such as that in panel (a), are encouraged.

• We don't know the answer, but there are two possibilities:

- 1) Network initialization may encourage smoothness, i.e. the model never departs from the sub-domain of smooth functions during training.
- 2) The training algorithm may somehow "prefer" to converge to smooth functions. Any factor that biases a solution towards a subset of equivalent solutions is known as a regularizer. So maybe training algorithms act as an implicit regularizer.

8.5: Choosing Hyperparameters:

- In the classical regime, we don't have access to the bias [which requires knowledge of the true underlying function] or the variance [which requires multiple independently sampled datasets to estimate]
- In the modern regime, there is no way to tell how much capacity should be added before the test error stops improving.
- So how should we choose model capacity in practice?
- The process of finding the best hyperparameters is called hyperparameter search / neural architecture search.
- Hyperparameters are chosen empirically, through experimentation.
 - ↳ We keep the best performing model.
 - ↳ But we don't measure performance on the test set [could be possible that the model doesn't generalize past the test set]
 - ↳ Instead, we use a third dataset called the validation set.
- ↳ So for each set of hyperparameters:
 - 1) Train the model on the training set
 - 2) Test on the validation set

3) Take the model that works best on the validation set, ; test it on the testing set.

* This recipe gives us a reasonable estimate on true performance.

8.6: Summary:

- To measure performance, we use a test set.
- The degree to which performance is maintained on this test set is known as generalization.
- Test errors can be explained by:
 - 1) Noise
 - 2) Bias
 - 3) Variance
- Adding training data decreases variance
- When the model capacity is less than the number of training examples, increasing capacity decreases bias but increases variance.
 - ↳ Bias, variance trade-off
 - ↳ There is a capacity where the trade off is optimal.
- This is balanced with a tendency for performance to improve with more model capacity.
 - ↳ The 2 phenomena give us the double descent curve.
 - ↳ It's believed that the model interpolates smoothly between training data in the modern regime. We don't know why.
- To choose model capacity ; other hyperparameters, we train multiple models ; evaluate performance with a separate validation set.
- See dar et al. for survey on theories of overparametrized models